1. Grzegorz Grudziński, 2. Szymon Karpiński
Politechnika Warszawska, Wydział Mechatroniki

Algorytm autonomicznego sterowania dronem

Streszczenie. Raport przedstawia projekt, którego głównym celem było zaprojektowanie i przetestowanie autonomicznego systemu sterowania dronem po wyznaczonej trasie.

Abstract. This report presents a project whose primary objective was to design and test an autonomous drone control system following a predefined path.

Słowa kluczowe. Dron, autonomiczny, Matlab, przetwarzanie obrazu, układ sterowania

Keywords. Drone, autonomus, Matlab, Image Processing System, Control System

Wstęp

W ostatnich latach obserwuje się znaczący wzrost wykorzystania bezzałogowych statków powietrznych. Znajdują one zastosowanie w wielu dziedzinach, takich jak przemysł militarny, logistyka, przemysł wydobywczy czy energetyka. Dzięki nim możliwe jest monitorowanie trudno dostępnych terenów, przyspieszenie dostaw, a także wsparcie działań ratowniczych i badawczych.
Autonomiczne systemy sterowania dronami odpowiadają za planowanie trajektorii, unikanie przeszkód oraz dostosowywanie się do zmieniających się warunków w czasie rzeczywistym. Takie rozwiązania zwiększają wydajność pracy, umożliwiając dronom wykonywanie zadań w sposób samodzielny i ciągły, bez konieczności stałego nadzoru operatora. Pozwala to na realizację misji w trudnych lub niebezpiecznych warunkach, oszczędność czasu oraz lepsze wykorzystanie zasobów ludzkich.

Cel projektu

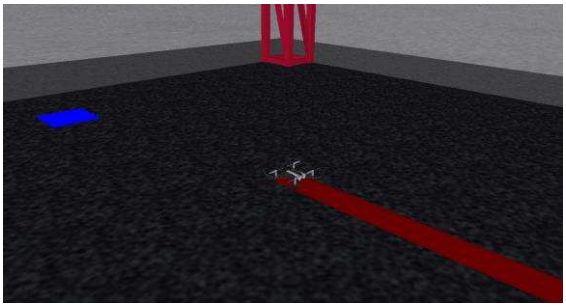
Celem projektu jest opracowanie i przetestowanie prostego algorytmu autonomicznego sterowania dronem po wyznaczonej trasie. Testy obejmują zarówno symulację komputerową, jak i rzeczywiste działanie algorytmu na dronie, co pozwala ocenić skuteczność zaproponowanego rozwiązania w praktyce.
Zgodnie z wytycznymi konkursu Minidrone Competition firmy MathWorks, projekt ten wpisuje się w założenia podejścia Model-Based Design z wykorzystaniem środowiska Simulink oraz opracowania algorytmu autonomicznego śledzenia linii dla quadkoptera. [1]

Opis środowiska pracy

Do realizacji projektu wykorzystano symulator udostępniony przez firmę MathWorks. Zawiera on model fizyki drona oraz środowisko do projektowania tras i przeprowadzania symulacji lotu. Dzięki temu możliwe jest testowanie algorytmu w kontrolowanych warunkach, bez ryzyka uszkodzenia rzeczywistego sprzętu, co pozwala na bezpieczne sprawdzenie skuteczności i stabilności opracowanego systemu sterowania.
W celu uruchomienia symulacji należy pobrać następujące rozszerzenia:

- Aerospace Toolbox
- Aerospace Blockset
- Computer Vision Toolbox
- Control System Toolbox
- DSP System toolbox
- Embedded Coder
- Image Processing Toolbox
- MATLAB Coder
- Motor control blockset
- Optimization Toolbox
- Signal Processing Toolbox
- Simulink 3D Animation
- Simulink Coder
- Simulink Design Optimization
- Stateflow
- System Identification Toolbox
- UAV toolbox

Następnie po uruchomieniu środowiska Matlab, symulację inicjuje się poprzez komendę:
parrotMinidroneCompetitionStart.



Rys.1 Obraz z symulatora

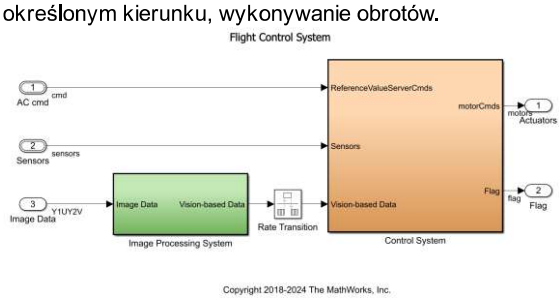
Wykorzystany model drona

Do przeprowadzania testów zarówno w środowisku symulacyjnym, jak i w rzeczywistych warunkach wykorzystano drona Parrot Mambo.

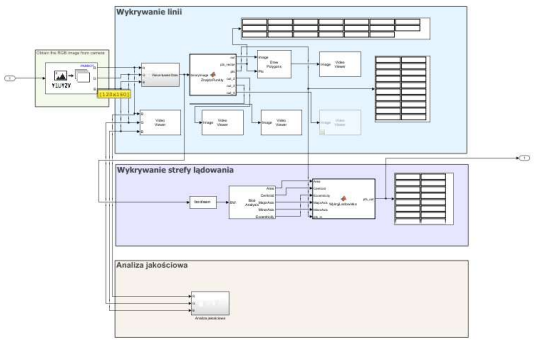
Rys. 2 Dron Parrot Mambo

Rozpoczęcie pracy nad algorytmem

Projektowanie algorytmu odbywa się w bloku Flight Control System, który składa się z dwóch głównych modułów: Image Processing System oraz Control System. Głównym zadaniem modułu Image Processing System jest wykrywanie trasy poprzez rozpoznanie koloru i binaryzację obrazu. Binaryzacja umożliwia dalsze przetwarzanie obrazu przy użyciu operacji morfologicznych, takich jak szkieletyzacja czy erozja. Dodatkowo moduł ten odpowiada za wykrywanie miejsca do lądowania poprzez rozpoznawanie kształtu koła, które reprezentuje lądowisko. W module Control System, realizowanym w środowisku Stateflow, znajduje się całe sterowanie dronem. Jego głównym zadaniem jest przetwarzanie danych pochodzących z modułu Image Processing System oraz kontrola lotu drona, w tym planowanie trasy, ruch w określonym kierunku, wykonywanie obrotów.



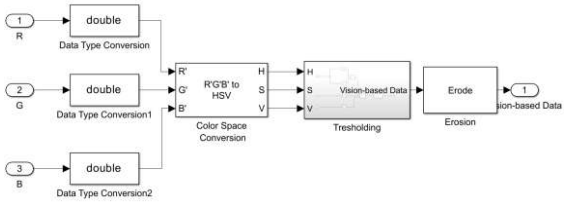
Rys. 3 Flight Control System



Rys. 3.1 Struktura wykrywania obrazu

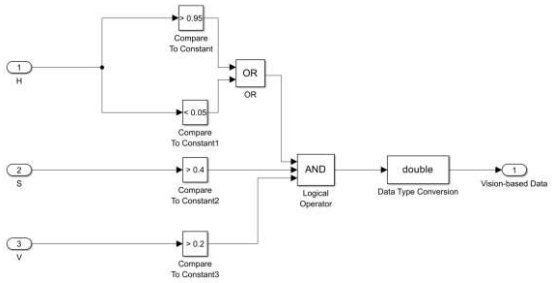
Binaryzacja obrazu

Proces binaryzacji obrazu został zrealizowany w module Image Processing System. W pierwszym etapie obraz z kamery pokładowej drona jest przekształcany do przestrzeni HSV za pomocą bloku *Color Space Conversion*. Zastosowanie przestrzeni HSV (Hue – barwa, Saturation – nasycenie, Value – jasność) pozwala na łatwiejsze rozróżnianie koloru trasy niezależnie od zmian oświetlenia, ponieważ składowa barwy (H) jest oddzielona od jasności obrazu (V).Następnie przeprowadzany jest proces progowania (thresholding). Wartości **H**, **S** oraz **V** są porównywane z zadanymi wartościami progowymi w celu wydzielenia z obrazu koloru czerwonego, zgodnie z kołem barw przestrzeni HSV. Umożliwia to selektywne wykrywanie trasy na podstawie jej koloru. W przypadku zmiany koloru trasy nie jest konieczna modyfikacja struktury algorytmu – wystarczy jedynie dostosować wartości progowe składowych HSV. Piksele spełniające zadane warunki progowe są klasyfikowane jako element trasy, natomiast pozostałe są odrzucane. W wyniku tego procesu otrzymywany jest obraz binarny. Uzyskana binaryzacja stanowi podstawę do dalszego przetwarzania obrazu, w tym wykonywania operacji morfologicznych, takich czy szkieletyzacja trasy.



Rys. 4 Binaryzacja obrazu

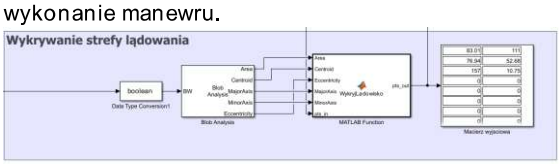
Wartości stałych zostały dobrane tak, aby w procesie binaryzacji wyodrębniana była wyłącznie trasa przelotu drona, a pozostałe elementy obrazu były ignorowane.



Rys. 5 Porównywanie wartości HSV

Wykrywanie strefy lądowania

Strefa lądowania w symulacji zawsze ma kształt koła, co stanowi istotną informację wykorzystywaną do rozpoznawania obiektu. Do detekcji zastosowano blok Blob Analysis z funkcją Eccentricity, która mierzy stopień eliptyczności wykrytego obiektu. W przypadku braku okrągłego obiektu w polu widzenia kamery wartość eccentricity waha się w zakresie od 0,7 do 1. W momencie pojawienia się obiektu o kształcie koła wartość ta spada, i wyjście landing zwraca aktualną wartość Eccentricity. Warunkiem przejścia do stanu lądowania jest osiągnięcie przez tę wartość progu 0,2, co powoduje precyzyjne wykonanie manewru.

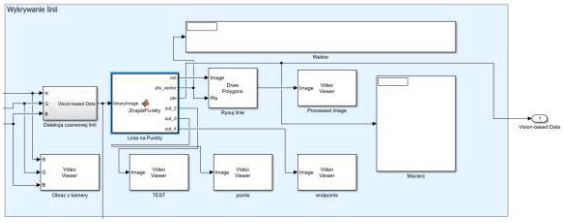


Rys 6. Wykrywanie strefy lądowania

Wykrywanie trasy

Algorytm wykrywania punktów trasy drona działa na binaryzowanym obrazie wejściowym i składa się z kilku etapów. Na początku obraz poddawany jest szkieletyzacji przy użyciu funkcji bwmorph z parametrem 'skel', co pozwala wyodrębnić centralną linię trasy, redukując grubość linii do jednego piksela i eliminując zbędne fragmenty. Następnie stosowana jest operacja branchpoints, która oznacza miejsca, gdzie linia rozgałęzia się(końce linii po operacji 'skel'), zamieniając linię na punkty początku i końca prostej.
Punkty filtrowane są w oparciu o algorytm regresji liniowej – wyznaczana jest linia prosta między początkiem a końcem segmentu i mierzone jest maksymalne odchylenie punktów pośrednich od tej linii. Jeśli błąd dopasowania przekracza ustalony próg, punkt ten klasyfikowany jest jako wierzchołek zakrętu, co skutecznie eliminuje szum i zapobiega wykrywaniu fałszywych łuków na prostych odcinkach. W algorytmie stosowany jest limit maksymalnej liczby punktów oraz wektora wyjściowego, co zapewnia stabilność działania i kontrolę pamięci.

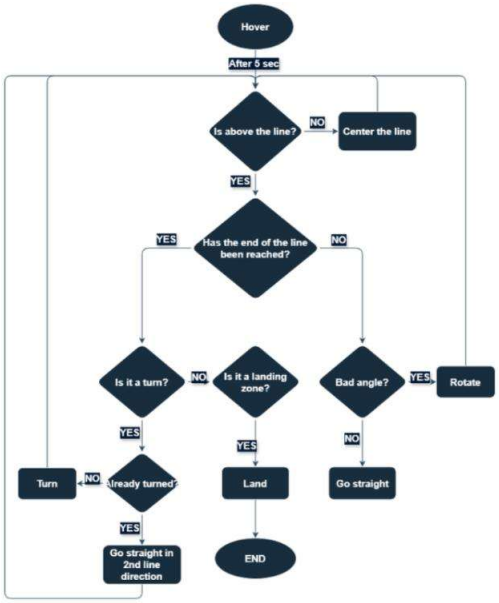
Ostatnim etapem jest sortowanie geometryczne wybranych punktów wzdłuż linii trasy. Przy trzech punktach identyfikowane są punkty skrajne i środkowy, aby uzyskać spójną kolejność od początku do końca trasy.



Rys. 7 Model wykrywania linii w środowisku Simulink

Logika układu sterowania

Lot drona realizowany jest w oparciu o stanowy algorytm sterowania, który określa kolejne działania w zależności od położenia drona względem trasy. Na początku dron znajduje się w stanie **hover**, utrzymując stabilną pozycję przez kilka sekund. Następnie system sprawdza, czy dron znajduje się nad linią trasy. Jeśli nie, dron centruje się nad **trasą** i obraca zgodnie z jej kierunkiem, aby rozpocząć lot wzdłuż linii.
Gdy dron znajduje się nad trasą, sprawdzane jest, czy osiągnął koniec segmentu. Jeśli nie, algorytm porównuje aktualny kąt lotu z kierunkiem segmentu i w razie potrzeby koryguje obrót, a w przeciwnym wypadku dron kontynuuje lot prosto wzdłuż segmentu.
Po dotarciu do końca segmentu dron sprawdza, czy występuje zakręt. W takim przypadku obraca się zgodnie z wykrytym kątem i rozpoczyna lot wzdłuż kolejnego segmentu. Proces powtarza się aż do momentu, gdy dron osiągnie koniec trasy i jednocześnie wykryje strefę lądowania w postaci okręgu. W tym momencie dron rozpoczyna procedurę lądowania.



Rys 8. Schemat blokowy logiki lotu

Opis układu sterowania w środowisku Stateflow

Układ sterowania został zrealizowany w środowisku Stateflow. Umożliwia to łatwy podział działania na stany, co znacząco ułatwia projektowanie takiego systemu. Układ działa na podstawie danych zebranych i przetworzonych w bloku Image Processing System.

Blok **Hover** odpowiada za początkową fazę lotu, w której dron wznosi się i utrzymuje stabilną pozycję nad punktem startowym.

Stan **FindTheState** odpowiada za podejmowanie decyzji dotyczących aktualnego sposobu lotu drona na podstawie danych geometrycznych otrzymanych z bloku przetwarzania obrazu. Na wejściu wykorzystywane są współrzędne trzech punktów opisujących wykrytą trasę (P1, P2, P3), położenie drona w obrazie oraz informacje o orientacji linii lotu. Na początku sprawdza, czy trasa jest widoczna, w przypadku braku wykrytych punktów aktywowany jest tryb oczekiwania. Następnie analizowane jest położenie drona względem linii trasy jeżeli dron nie znajduje się nad linią, uruchamiany jest tryb jej ponownego odnalezienia i wycentrowania.

Gdy dron znajduje się poprawnie nad trasą, wykonywana jest analiza dalszego lotu. Sprawdzane jest, czy dron zbliżył się do końca aktualnego odcinka trasy oraz czy przed nim występuje zakręt. W przypadku wykrycia zakrętu system przechodzi do stanu zatrzymania i obrotu zgodnie z wykrytym kątem. Jeżeli trasa jest prosta, dron kontynuuje lot w kierunku kolejnego punktu. Dodatkowo na bieżąco kontrolowany jest kąt ustawienia drona względem linii – w razie potrzeby wykonywana jest korekta orientacji. Na podstawie wybranego punktu docelowego wyznaczany jest błąd w osi podłużnej, który wykorzystywany jest do dalszego sterowania ruchem drona.

Stan **FindTheLine** służy do wycentrowania drona nad linią. Na podstawie danych z kamery wyznaczany jest błąd położenia drona względem środka obrazu w osiach X oraz Y. Następnie obliczane są niewielkie prędkości korekcyjne, ograniczone do bezpiecznych wartości, które przesuwają drona w kierunku linii.
Prędkości te są przeliczane na układ globalny z uwzględnieniem aktualnego kąta obrotu drona, a pozycja

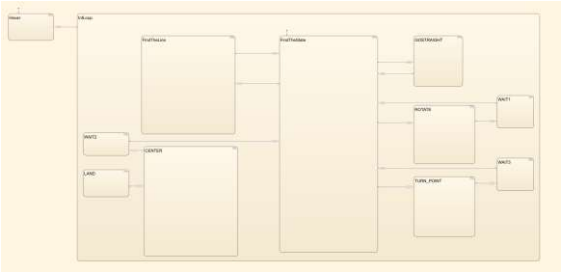
jest stopniowo aktualizowana przy zachowaniu stałej wysokości lotu. Po osiągnięciu położenia bliskiego środka trasy ustawiana jest flaga zakończenia centrowania, co pozwala na przejście do dalszego etapu lotu.

Stan **GOSTRAIGHT** odpowiada za lot drona wzdłuż wyznaczonej trasy. Steruje on prędkością drona w kierunku ruchu z uwzględnieniem aktualnego kąta obrotu, zachowując stałą wysokość.

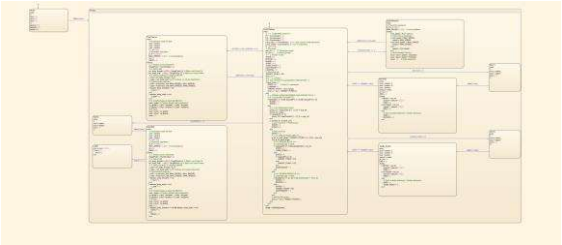
Blok **ROTATE** odpowiada za precyzyjne obracanie drona w miejscu w celu ustawienia odpowiedniego kąta względem aktualnej linii. Dron zatrzymuje ruch w przód, w bok i w pionie, a następnie dostosowuje kąt obrotu w małych krokach, aż różnica kąta znajdzie się w dopuszczalnej strefie tolerancji.

Stan **TURN_POINT** służy do wykonania obrotu drona w miejscu przy zakręcie trasy. Dron wstrzymuje ruch w przód, w bok i w pionie, a następnie stopniowo koryguje kąt (yaw) zgodnie z wykrytym kątem zakrętu (Alpha2). Obrót trwa, aż dron znajdzie się w dopuszczalnej strefie tolerancji kąta, po czym sygnał obrotu jest wyłączany.

LAND odpowiada za końcową fazę lotu. Po przejściu do tego bloku dron zaczyna stopniowo obniżać się aż do osiągnięcia wyznaczonego miejsca do lądowania.



Rys 9. Układ sterowania wykonany w Stateflow

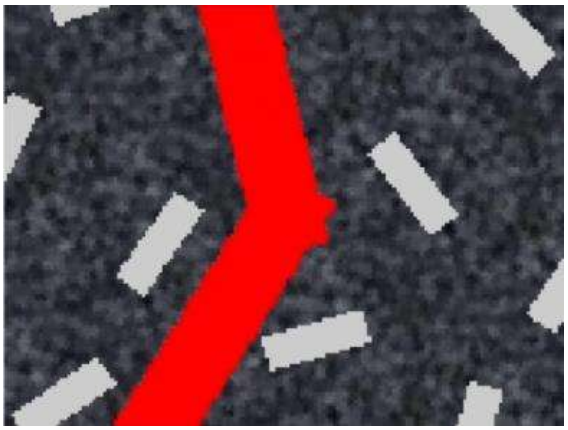


Rys 9.1. Układ sterowania wykonany w Stateflow (z kodem)

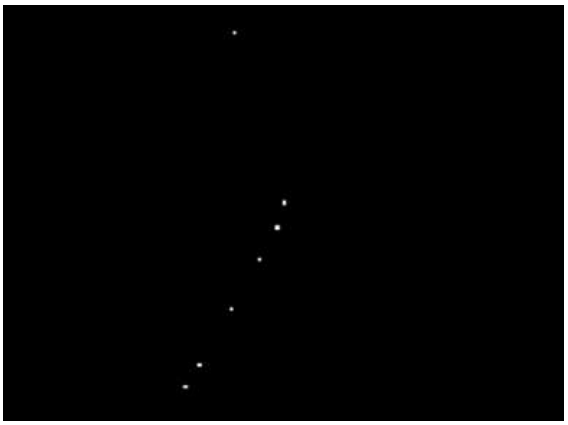
Rezultaty

Opracowane rozwiązanie pozwala drowi na autonomiczną nawigację wzdłuż wyznaczonej trasy. System poprawnie realizuje zadania śledzenia linii oraz wykonywania manewrów skrętu, choć zauważono, że przy bardziej złożonych operacjach, takich jak zakręt, dron wymaga dłuższego czasu na ustabilizowanie pozycji. Istotnym wyzwaniem w procesie detekcji okazały się zakręty o kątach innych niż 90 stopni, które generują zwiększony poziom szumu w danych wizyjnych. Mimo zastosowania filtracji, czasami występuje zjawisko "skakania" wykrywanych punktów, co wpływa na płynność ruchu urządzenia.

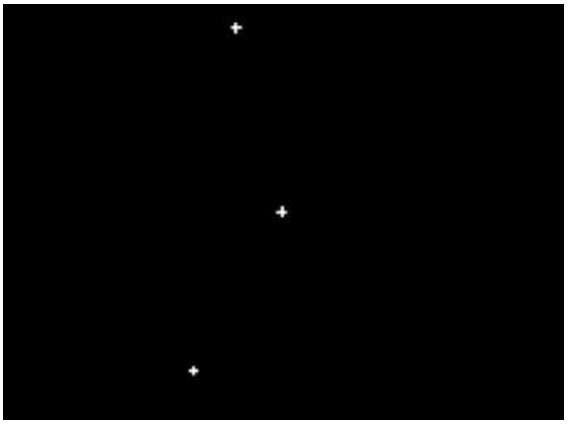
Zastosowana architektura modułowa znacząco ułatwia dalszy rozwój projektu. Dzięki separacji warstwy przetwarzania obrazu, zwracającej współrzędne punktów od logiki sterowania, możliwa jest niezależna optymalizacja algorytmów filtracji bez ingerencji w system lotu. Ponadto, wykorzystanie maszyny stanów do obsługi poszczególnych faz lotu pozwala na precyzyjne wprowadzanie poprawek do konkretnych manewrów drona.



Rys 10. Obraz z kamery drona



Rys 11. Wykrywane punkty przed filtracją



Rys 12. Wykrywane punkty po filtracji

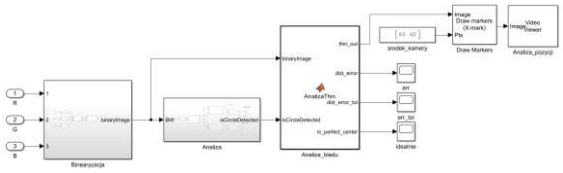
Analiza Jakościowa

W celu weryfikacji precyzji sterowania zaimplementowano moduł Analizy Jakościowej. Działa on niezależnie od głównego programu, monitorując błąd pozycjonowania drona względem trasy w czasie rzeczywistym.

Odczytywane dane błędów pozycji zapisywane są do plików z rozszerzeniem ".mat" w katalogu roboczym projektu:

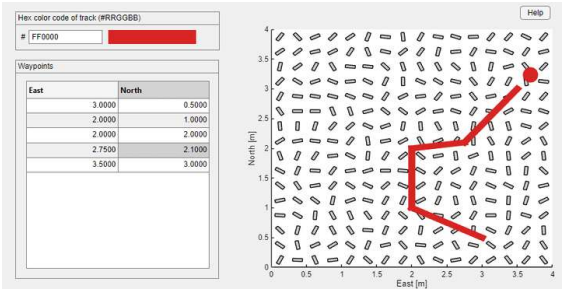
- error.mat - błąd pozycji bez tolerancji
- error_tolerance.mat - błąd pozycji z tolerancją
- landing_err.mat - błąd pozycji przy lądowaniu

Proces przetwarzania danych przebiega w kilku etapach:
1. Wstępne przetwarzanie: obraz jest binearyzowany, aby badać wyłącznie czerwoną linię i strefę lądowania
2. Analiza: Głównym elementem jest blok Analiza_bledu, wykonujący operację szkieletonizacji linii, a następnie liczy różnicę położenia środka kamery od obiektu. Analogicznie działa blok analizy strefy lądowania.
3. Wizualizacja: Wyniki liczbowe rejestrowane są na oscyloskopach (`err`, `err_tol`, `landing_err`), natomiast pozycja linii jest wizualizowana na podglądzie wideo w bloku **Analiza_pozycji** za pomocą znacznika naniesionego na obraz.



Rys 13. Trasa testowa

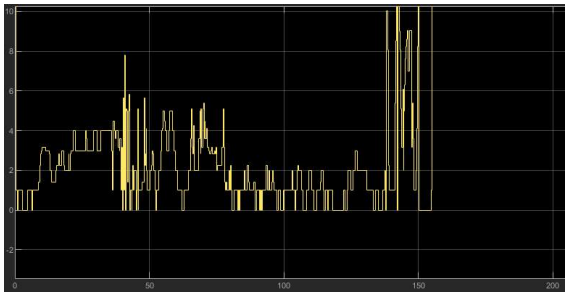
Testy przeprowadzono na trasie odcinki proste, zakręty 90° oraz lądowisko. Taka konfiguracja weryfikuje poprawność detekcji linii oraz działania algorytmu lotu. (Aby odtworzyć trasę, należy w środowisku Matlab wybrać opcję *"Open track builder"*, a następnie wpisać punkty jak na rys. 14)



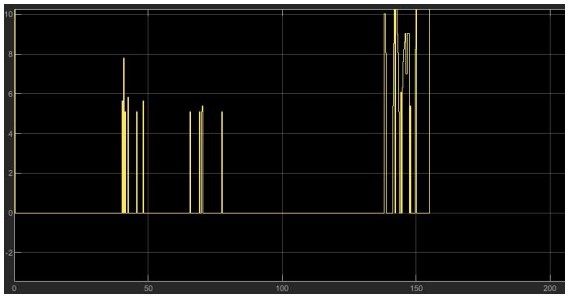
Rys 14. Trasa testowa

Dron przelatuje trasę w czasie 2:31. Najwięcej czasu zajmuje pokonanie zakrętów, gdzie pojawia się największy szum na obrazie, oraz gdzie dron musi ustabilizować swoją pozycję i zmienić swoją orientację i kierunek lotu.

Na rysunkach 15 i 16 przedstawiono wykresy błędów pozycji środka drona od środka linii (w pikselach). Wykres 16 przedstawia surowy błąd między środkiem kamery a linią bez uwzględnienia zadanej tolerancji, wykres 17 pokazuje wartości po uwzględnieniu tolerancji. Na wykresach widać, że błąd oscyluje głównie między wartościami do 4 px, przy zadanej tolerancji 5 px. Największe błędy pojawiają się w sytuacjach takich jak zakręt i podejście do lądowania, gdzie dron musi wykonać bardziej skomplikowane manewry. Niewielkie odchyłki potwierdzają, że dron skutecznie podąża za trasą, mimo naturalnych oscylacji.

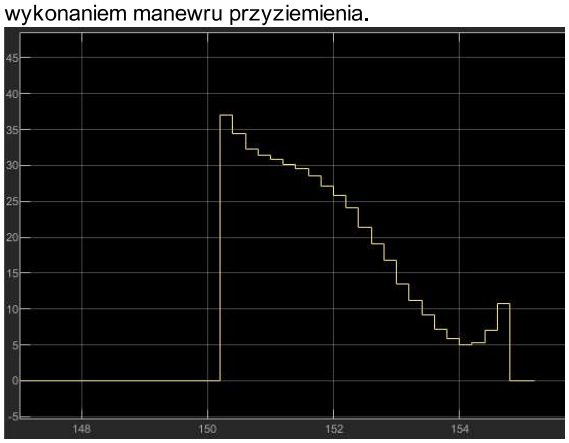


Rys 15. Błąd pozycji bez uwzględnienia tolerancji



Rys 16. Błąd pozycji z uwzględnieniem tolerancji

Wykres na rysunku 17 obrazuje błąd odległości od środka lądowiska. Wyraźny trend spadkowy do zera potwierdza skuteczność algorytmu centrującego tuż przed wykonaniem manewru przyziemienia.



Rys 17. Błąd pozycji przy lądowaniu

Wnioski i kolejne kroki

Realizacja projektu pozwoliła na skuteczne opracowanie i przetestowanie algorytmu autonomicznego sterowania dronem Parrot Mambo. Osiągnięto główny cel projektu, jakim było stworzenie systemu umożliwiającego autonomiczny lot po wyznaczonej trasie oraz lądowanie.

Analizując uzyskane wyniki w odniesieniu do innych prac w tej tematyce, warto przywołać systematyczny przegląd autorstwa Katkuri i in. (2024) [2]. Autorzy ci wskazują na rosnącą dominację metod opartych na głębokim uczeniu (Deep Learning) i splotowych sieciach neuronowych (CNN) w zadaniach nawigacji UAV. Mimo że metody te oferują wysoką generalizację w złożonych środowiskach, wiążą się one ze znacznym zapotrzebowaniem na moc obliczeniową oraz koniecznością posiadania obszernych zbiorów uczących. W tym kontekście, zrealizowany w niniejszej pracy system, oparty na klasycznych metodach

wizji komputerowej (takich jak operacje morfologiczne, szkieletyzacja czy analiza blobów) oraz deterministycznej maszynie stanów, stanowi istotną alternatywę dla systemów wbudowanych o ograniczonych zasobach. Wykazano, że dla zdefiniowanych zadań (śledzenie linii, lądowanie na znaczniku) podejście algorytmiczne zapewnia wystarczającą precyzję i szybkość działania przy ułamku kosztu obliczeniowego wymaganego przez zaawansowane modele AI. Potwierdza to, że w zastosowaniach na lekkich i tanich dronach, optymalizacja klasycznych algorytmów wciąż może odgrywać kluczową rolę.

Perspektywa dalszych prac obejmuje przede wszystkim udoskonalenie warstwy przetwarzania obrazu, w szczególności poprzez optymalizację algorytmów filtracji, co pozwoli na redukcję zakłóceń i zwiększenie stabilności detekcji trasy, a co za tym idzie płynności lotu. Istotnym kierunkiem rozwoju jest również ewentualna implementacja sterownika wykorzystującego logikę rozmytą, który mógłby znacząco wpłynąć na płynność korekcji toru lotu. Działania te, w połączeniu z dalszym dostrajaniem dynamiki drona stanowią niezbędny etap przygotowań do przeniesienia opracowanego rozwiązania ze środowiska symulacyjnego na rzeczywiste urządzenie i przeprowadzenie testów w warunkach realnych.

Bibliografia

[1] www.mathworks.com/content/dam/mathworks/mathworks-dot-com/academia/student-competitions/minidrone-competition/mathworks-minidrone-competition-guidelines.pdf
[2] Aditya Vardhan Reddy Katkuri et al., Autonomous UAV navigation using deep learning-based computer vision frameworks: A systematic literature review, Science Direct, 23 (2024)
[3] www.mathworks.com/academia/students/competitions/minidrones.html
[4] www.mathworks.com/videos/programming-drones-with-simulink-1513024653640.html
[5] www.mathworks.com/content/dam/mathworks/mathworks-dot-com/company/events/webinar-cta/3984200-Presentation.pdf